

Digital Engineering

Introduction

Digital Computer Systems

- Digital systems consider *discrete* amounts of data
- Examples
  - 26 letters in the alphabet
  - 10 decimal digits
- Larger quantities can be built from discrete values:
  - Words made of letters
  - Numbers made of decimal digits (e.g. 239875.32)
- Computers operate on binary values (0 and 1)
- Easy to represent binary values electrically
  - Voltages and currents
  - Can be implemented using circuits
  - Create the building blocks of modern computers

Understanding Decimal Numbers

- **Decimal numbers are made of decimal digits: (0,1,2,3,4,5,6,7,8,9) ⇒ Base = 10**
- **But how many items does a decimal number represent?**
  - **$8653 = 8 \times 10^3 + 6 \times 10^2 + 5 \times 10^1 + 3 \times 10^0$**
  - **Number = $d_3 \times B^3 + d_2 \times B^2 + d_1 \times B^1 + d_0 \times B^0$ = Value**
- **What about fractions?**
  - **$97654.35 = 9\times10^4 + 7\times10^3 + 6\times10^2 + 5\times10^1 + 4\times10^0 + 3\times10^{-1} + 5\times10^{-2}$**
  - **In formal notation → $(97654.35)_{10}$**

Understanding Binary Numbers

- **Binary numbers are made of binary digits (bits):**
  - **0 and 1**
- **How many items does a binary number represent?**
  - **8 4 2 1 = Weights**
  - $(1011)_2 = 1x2^3 + 0x2^2 + 1x2^1 + 1x2^0 = (11)_{10}$
- **What about fractions?**
  - $(110.10)_2 = 1x2^2 + 1x2^1 + 0x2^0 + 1x2^{-1} + 0x2^{-2}$
- **Groups of eight bits are called a byte**
  - $(11001001)_2$
- **Groups of four bits are called a nibble**
  - $(1101)_2$

Understanding Octal Numbers

- **Octal numbers are made of octal digits: (0,1,2,3,4,5,6,7)**
- **How many items does an octal number represent?**
  - **512 64 8 1 = Weights**
  - **$(4536)_8 = 4x8^3 + 5x8^2 + 3x8^1 + 6x8^0 = (2398)_{10}$**
- **What about fractions?**
  - **$(465.27)_8 = 4x8^2 + 6x8^1 + 5x8^0 + 2x8^{-1} + 7x8^{-2}$**
- **Octal numbers don't use digits 8 or 9**

Understanding Hexadecimal Numbers

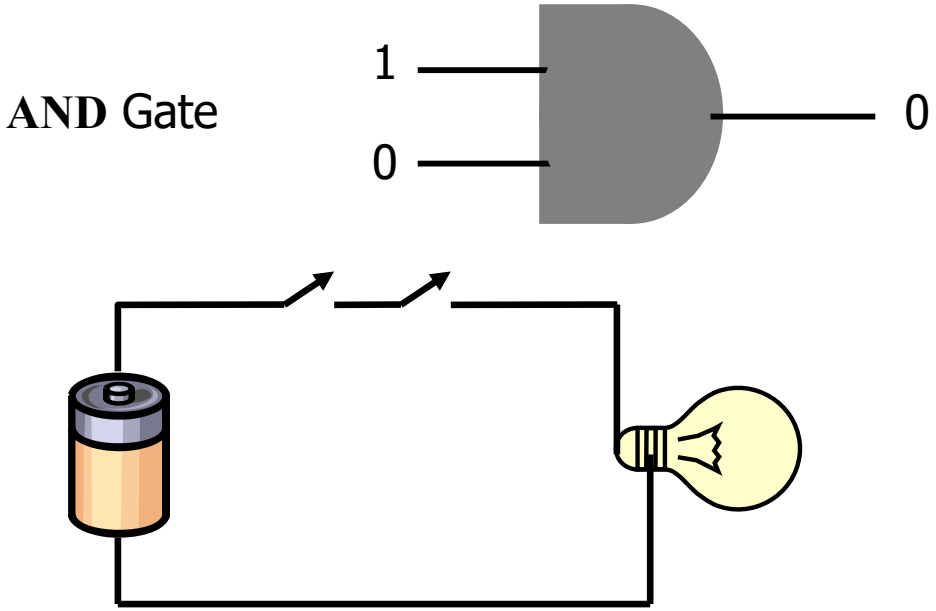
- **Hexadecimal numbers are made of 16 digits:**
  - **(0,1,2,3,4,5,6,7,8,9,A, B, C, D, E, F)**
- **How many items does a hex number represent?**

4096 256 16 1 = Weights

  - $(3A9F)_{16} = 3x16^3 + 10x16^2 + 9x16^1 + 15x16^0 = 15007_{10}$
- **What about fractions?**
  - $(2D3.5)_{16} = 2x16^2 + 13x16^1 + 3x16^0 + 5x16^{-1} = 723.3125_{10}$
- **Note that each hexadecimal digit can be represented with four bits**
  - $(1110)_2 = (E)_{16}$

Why Use Binary Numbers?

- **Easy to represent 0 and 1 using electrical values**
- **Possible to tolerate noise**
- **Easy to transmit data**
- **Easy to build binary circuits**



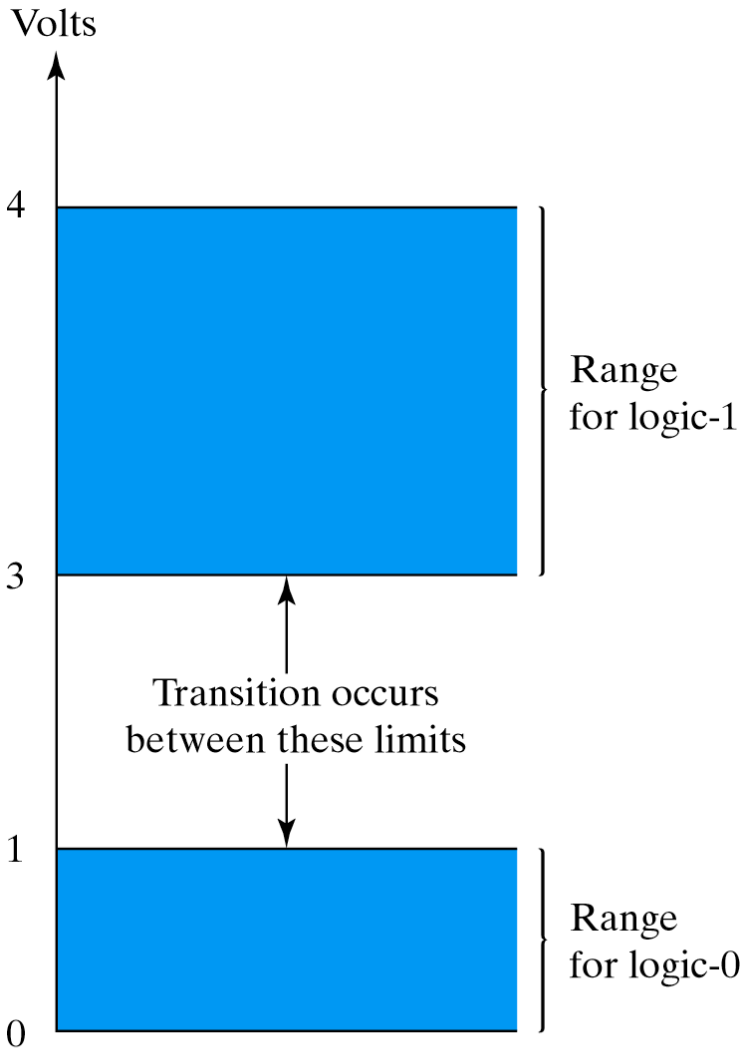


Fig. 1-3 Example of binary signals

Convert an Integer *from* Decimal *to* Another Base

For each digit position:

1. **Divide decimal number by the base (e.g. 2)**
2. **The *remainder* is the lowest-order digit**
3. **Repeat first two steps until no *divisor* remains**

Example for $(13)_{10}$:

	Quotient		Remainder	Coefficient
13/2 =	6	,	1	$a_0 = 1$
6/2 =	3	,	0	$a_1 = 0$
3/2 =	1	,	1	$a_2 = 1$
1/2 =	0	,	1	$a_3 = 1$

Answer $(13)_{10} = (a_3\ a_2\ a_1\ a_0)_2 = (1101)_2$

MSB **LSB**

Convert a Fraction *from* Decimal *to* Another Base

For each digit position:

1. **Multiply decimal number by the base (e.g. 2)**
2. **The *integer* is the highest-order digit**
3. **Repeat first two steps until fraction becomes zero**

Example for $(0.625)_{10}$:

	Integer		Fraction	Coefficient
0.625 x 2 =	1	+	0.250	$a_{-1} = 1$
0.250 x 2 =	0	+	0.500	$a_{-2} = 0$
0.500 x 2 =	1	+	0	$a_{-3} = 1$

Answer $(0.625)_{10} = (0.a_{-1}\, a_{-2}\, a_{-3})_2 = (0.101)_2$

MSB LSB



Conversion Between Base 16 and Base 2

- **Conversion is easy!**

  Determine the 4-bit value for each hex digit
- **Note that there are 16 different values of four bits**
- **Easier to read and write in hexadecimal**
- **Representations are equivalent!**

$$3A9F_{16} = \underline{0011}\ \underline{1010}\ \underline{1001}\ \underline{1111}_2$$

3 A 9 F

The Growth of Binary Numbers

n	2^n
0	$2^0=1$
1	$2^1=2$
2	$2^2=4$
3	$2^3=8$
4	$2^4=16$
5	$2^5=32$
6	$2^6=64$
7	$2^7=128$

n	2^n	
8	$2^8=256$	
9	$2^9=512$	
10	$2^{10}=1024$	Kilo
11	$2^{11}=2048$	
12	$2^{12}=4096$	
20	$2^{20}=1M$	Mega
30	$2^{30}=1G$	Giga
40	$2^{40}=1T$	Tera

Understanding Binary Coded Decimal

- ***Binary Coded Decimal*** **(BCD) represents each decimal digit with four bits**

Ex. $\underbrace{0011}_{3}\ \underbrace{0010}_{2}\ \underbrace{1001}_{9} = 329_{BCD}$

- **This is NOT the same as** $001100101001_2 = 809_{10}$

Digit	BCD Code
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Putting It All Together

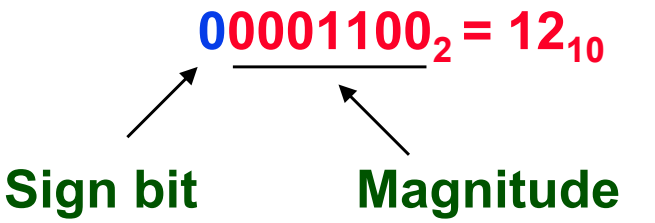
Decimal	Binary	Octal	Hexadecimal	BCD
0	0000	0	0	000
1	0001	1	1	0001
2	0010	2	2	0010
3	0011	3	3	0011
4	0100	4	4	0100
5	0101	5	5	0101
6	0110	6	6	0110
7	0111	7	7	0111
8	1000	10	8	1000
9	1001	11	9	1001
10	1010	12	A	0001 0000
11	1011	13	B	0001 0001
12	1100	14	C	0001 0010
13	1101	15	D	0001 0011
14	1110	16	E	0001 0100
15	1111	17	F	0001 0101

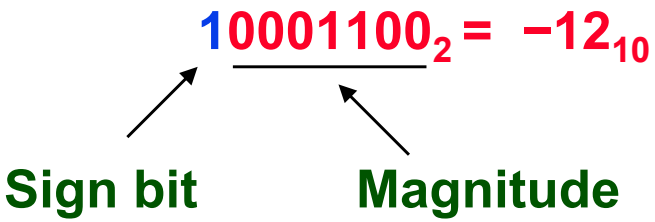
How To Represent Signed Numbers

- Plus and minus signs are used for decimal numbers:
  - 25 (or +25), −16, etc
- In computers, everything is represented as *bits*
- Three types of signed binary number representations:
  - signed magnitude
  - 1's complement
  - 2's complement
- In each case: left-most bit indicates the sign: '0' for positive and '1' for negative

Signed Magnitude Representation

- **The left most bit is designated as the *sign* bit while the remaining bits form the *magnitude***





One's Complement Representation

- **The one's complement of a binary number is done by complementing (i.e. inverting) all bits**

  1's comp of 00110011 is 11001100

  1's comp of 10101010 is 01010101

- **For a n-bit number N the 1's complement is $(2^n - 1) - N$**
- **Called "*diminished radix complement*" by Mano**
- **To find the negative of a 1's complement number take its 1's complement**



One's Complement Representation

4 bits

⇓

16 combinations

7	0111
6	0110
.	.
.	.
1	0001
0	0000
− 0	1111
− 1	1110
.	.
.	.
− 6	1001
− 7	1000

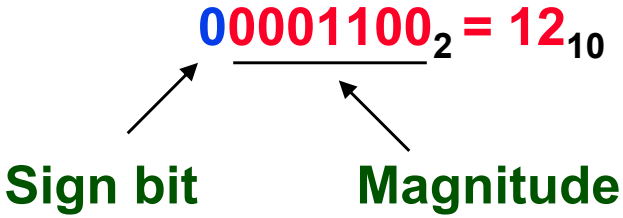
Two's Complement Representation

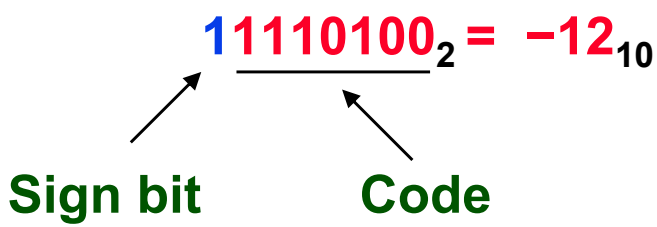
- **The two's complement of a binary number is done by complementing (inverting) all bits then adding 1**

  2's comp of 00110011 is 11001101

  2's comp of 10101010 is 01010110

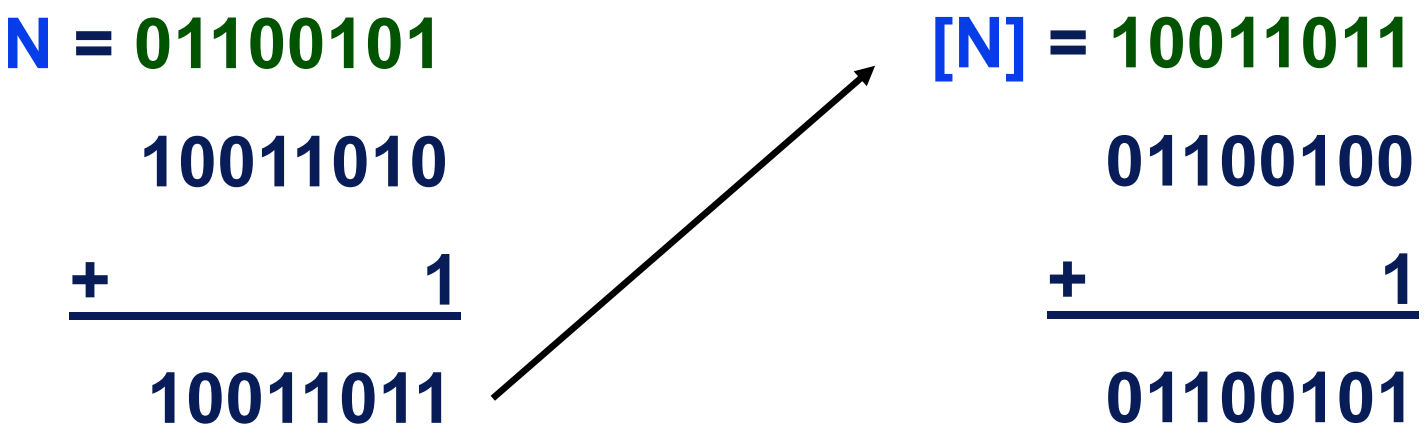
- **For an n-bit number N the 2's complement is $(2^n - 1) - N + 1$**
- **Called "*radix complement*" by Mano**
- **To find the negative of a 2's complement number take its 2's complement**





Two's Complement Shortcuts

- ***Algorithm 1***: **Complement each bit then add 1 to the result**



- ***Algorithm 2***: **Starting with the least significant bit, copy all of the bits up to and including the first '1' bit, then complement the remaining bits**

N = 0 1 1 0 0 1 1 0

[N] = 1 0 0 1 1 0 1 0

Two's Complement Representation

4 bits

⇓

16 combinations

7	0111
6	0110
.	.
2	0010
1	0001
0	0000
– 1	1111
– 2	1110
.	.
.	.
– 7	1001
– 8	1000

Putting All Together

Decimal	Signed-2's Complement	Signed-1's Complement	Signed Magnitude
+7	0111	0111	0111
+6	0110	0110	0110
+5	0101	0101	0101
+4	0100	0100	0100
+3	0011	0011	0011
+2	0010	0010	0010
+1	0001	0001	0001
+0	0000	0000	0000
−0	—	1111	1000
−1	1111	1110	1001
−2	1110	1101	1010
−3	1101	1100	1011
−4	1100	1011	1100
−5	1011	1010	1101
−6	1010	1001	1110
−7	1001	1000	1111
−8	1000	—	—

Finite-Precision Number Representation

- **Machines that use 2's complement arithmetic can represent integers in the range**

$$-2^{n-1} \le N \le 2^{n-1} - 1$$

n is the number of bits used for representing N

Note that $2^{n-1} - 1 = (011..11)_2$ and $-2^{n-1} = (100..00)_2$

- **2's complement code has more negative numbers than positive**
- **1's complement code has 2 representations for zero**
- **For an n-bit number in base (i.e. radix) z there are z^n different unsigned values (combinations)**

$$(0, 1, \ldots z^{n-1})$$

2's Complement Addition

- **Using 2's complement representation, adding numbers is easy**

  Step 1: Add binary numbers

  Step 2: Ignore the resulting carry bit

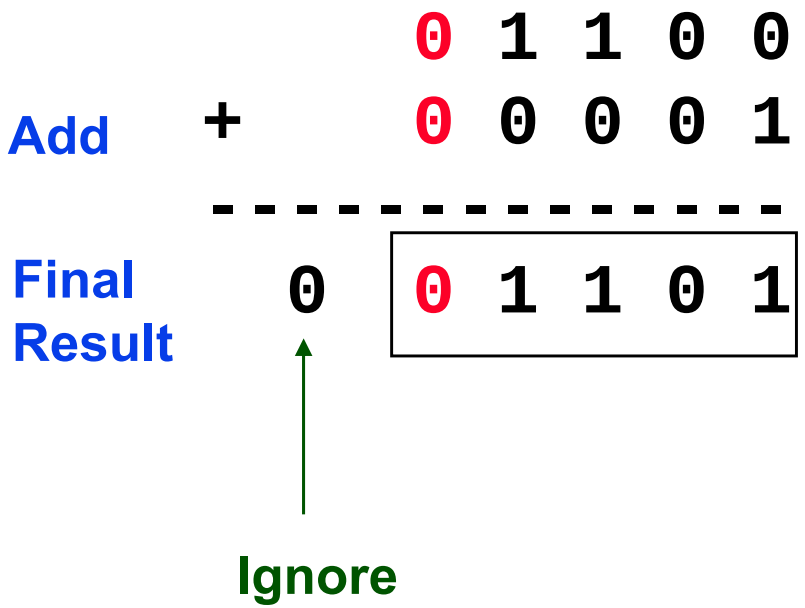
- **For example: $(12)_{10} + (1)_{10}$**

  $(12)_{10} = +(1100)_2$

  $= 01100_2$ *in 2's comp.*

  $(1)_{10} = +(0001)_2$

  $= 00001_2$ *in 2's comp.*



2's Complement Subtraction

- **Using 2's complement representation, subtracting numbers is also easy**

  Step 1: Take 2's complement of 2nd operand

  Step 2: Add binary numbers

  Step 3: Ignore the resulting carry bit

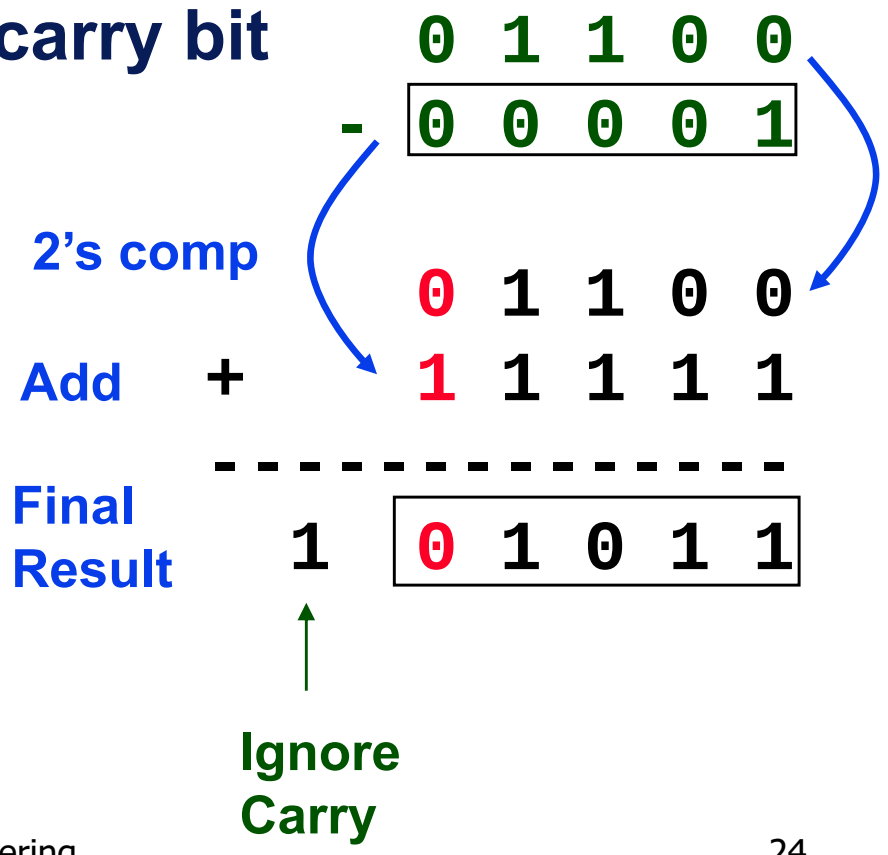
- **For example: $(12)_{10} - (1)_{10}$**

  $(12)_{10} = +(1100)_2$

  $= 01100_2$ *in 2's comp.*

  $(-1)_{10} = -(0001)_2$

  $= 11111_2$ *in 2's comp.*



2's Complement Subtraction (Cont'd)

- **Example 2: $(13)_{10} - (5)_{10}$**

  $(13)_{10} = +(1101)_2 = (01101)_2$

  $(-5)_{10} = -(0101)_2 = (11011)_2$

- **Adding these two 5-bit codes:**

$$\begin{array}{rr} & 0\ 1\ 1\ 0\ 1 \\ + & 1\ 1\ 0\ 1\ 1 \\ \hline \text{Carry} \longrightarrow \boxed{1} & 0\ 1\ 0\ 0\ 0 \end{array}$$

- **Discarding the carry bit, the sign bit is seen to be zero, indicating a positive result**

Indeed: $(01000)_2 = +(8)_{10}$

2's Complement Subtraction (Cont'd)

- **Example 3: $(5)_{10} - (12)_{10}$**

  $(5)_{10} = +(0101)_2 = (00101)_2$

  $(-12)_{10} = -(1100)_2 = (10100)_2$

- **Adding these two 5-bit codes:**

$$\begin{array}{rr} & 0\ 0\ 1\ 0\ 1 \\ + & 1\ 0\ 1\ 0\ 0 \\ \hline \text{Carry} \rightarrow & \boxed{0}\ 1\ 1\ 0\ 0\ 1 \end{array}$$

- **Here, there is no carry bit and the sign bit is 1. This indicates a negative result, which is what we expect: $(11001)_2 = -(7)_{10}$**

Gray Code

- Gray code is not a number system

  It is an alternate way to represent four bit data

- Only one bit changes from one decimal digit to the next
- Useful for reducing errors in communication
- Can be scaled to larger numbers

Digit	Binary	Gray Code
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

Text: ASCII Characters

- ASCII: Maps 128 characters to 7-bit code.
  - both printable and non-printable (ESC, DEL, …) characters
    http://www.asciitable.com/

00	nul	10	dle	20	sp	30	0	40	@	50	P	60	`	70	p
01	soh	11	dc1	21	!	31	1	41	A	51	Q	61	a	71	q
02	stx	12	dc2	22	"	32	2	42	B	52	R	62	b	72	r
03	etx	13	dc3	23	#	33	3	43	C	53	S	63	c	73	s
04	eot	14	dc4	24	$	34	4	44	D	54	T	64	d	74	t
05	enq	15	nak	25	%	35	5	45	E	55	U	65	e	75	u
06	ack	16	syn	26	&	36	6	46	F	56	V	66	f	76	v
07	bel	17	etb	27	'	37	7	47	G	57	W	67	g	77	w
08	bs	18	can	28	(	38	8	48	H	58	X	68	h	78	x
09	ht	19	em	29	)	39	9	49	I	59	Y	69	i	79	y
0a	nl	1a	sub	2a	*	3a	:	4a	J	5a	Z	6a	j	7a	z
0b	vt	1b	esc	2b	+	3b	;	4b	K	5b	[	6b	k	7b	{
0c	np	1c	fs	2c	,	3c	<	4c	L	5c	\	6c	l	7c	|
0d	cr	1d	gs	2d	-	3d	=	4d	M	5d	]	6d	m	7d	}
0e	so	1e	rs	2e	.	3e	>	4e	N	5e	^	6e	n	7e	~
0f	si	1f	us	2f	/	3f	?	4f	O	5f	_	6f	o	7f	del